



WEEK 10 LAB

MINING FOR MATERIALS

Welcome to the Week 10 lab. This lab is NOT Assessed like previous weeks but is instead provided as a sampling of additional techniques that could prove useful for the distinction and high distinction tasks.

This week we will be looking at setting up Master Materials, Material Instances, Parameter Collections

Learning outcomes this week:

- Understand the importance of master materials and material instances
- Using the material graph and parameters to make dynamic and updating materials through C++



1. Create a New Unreal Project

Create a New Unreal Project

Now you've got your repository set up, we can move onto creating our new project.

Launch Unreal Engine 5.7 and once the Project Browser pops up, select **Games** from the left side, and then select **Third Person**. With this selected, make sure you select the Project Defaults as follows:

Blueprint/C++: **C++**

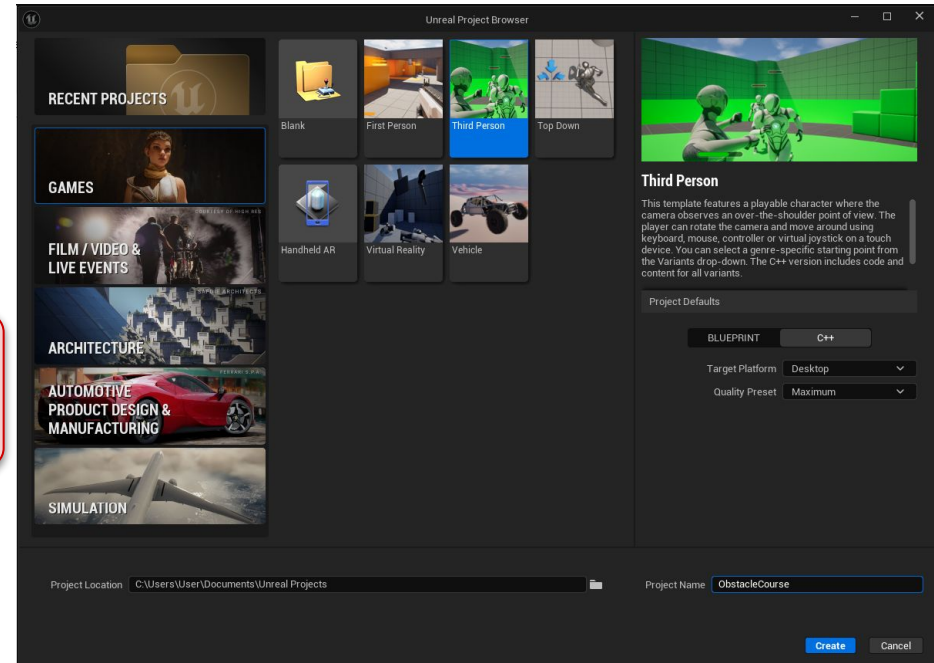
Target Platform: **Mobile**

Quality Preset: **Scalable**

Name your project **W10_TechArt** and select where you want to save your project.

Warning - Documents folder

Remember to save your work in the **local Documents folder** if you're on the lab machines, and ensure you **commit and push** your work at the end of the class, to ensure your work doesn't get wiped.



2. Importing Assets

Next we need to import the provided assets into our project. First, navigate to the **week 10 section on Moodle** and download the file **“Week 10 - Assets”**. Extract the files to the downloads folder then open the Unreal Editor.

Open the Content Browser, create a new Meshes folder then **Right Click -> Import to Current Folder**. Select **SM_Bread.fbx** and click Open. This will bring up the import window.

Change the following settings:

Import Materials: Unticked

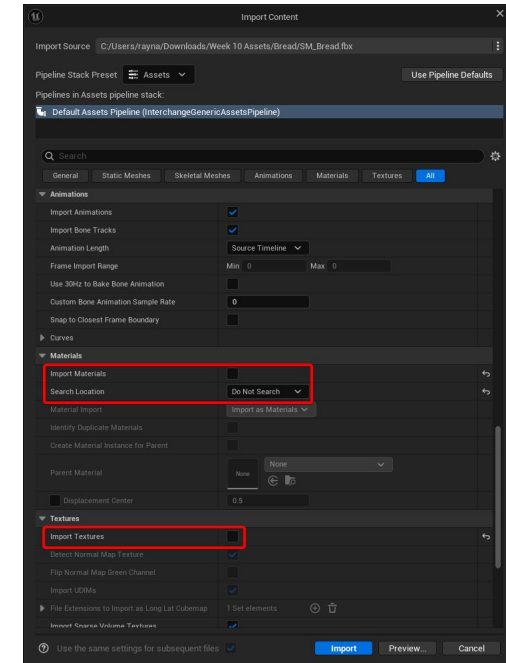
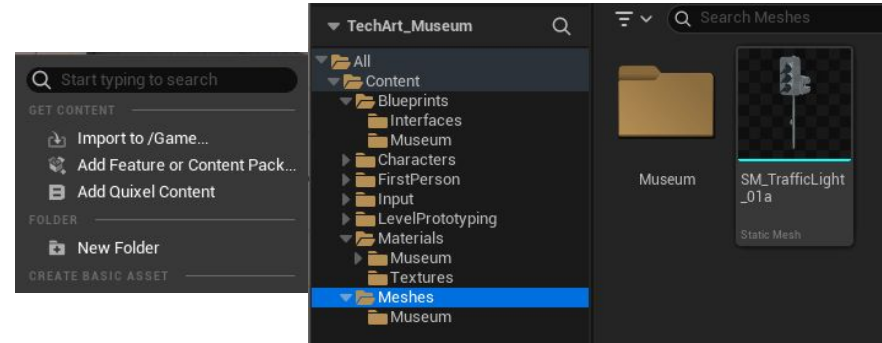
Search Location: Do Not Search

Import Textures: Unticked

Once done click **Import** to finish the process. Open the **Content Browser** again and navigate to the **Materials** folder. **Right Click** and select **Create New Folder**, naming it **“Textures”**. Open the folder and **import** all the provided **textures** into the folder.

Assets appearing too small?

For the bread asset we do not need to adjust its Uniform Scale. If you ever import a model and it is tiny, reimport the asset with Uniform Scale set to 100. Often this is needed for assets created with Unity in mind.



3. Fixing Textures

As mentioned in the first slide providing an overview of the tasks, our artists have been using slightly different settings to the standard Unreal Engine setup. Because of this we need to make sure that adjust the settings of the textures in-engine so that they align with what Unreal needs.

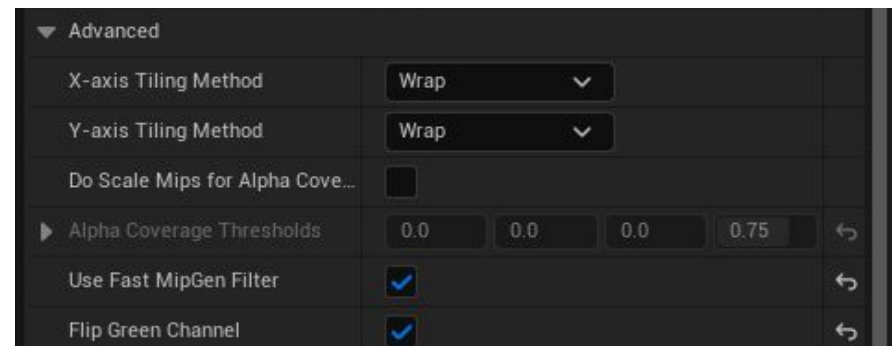
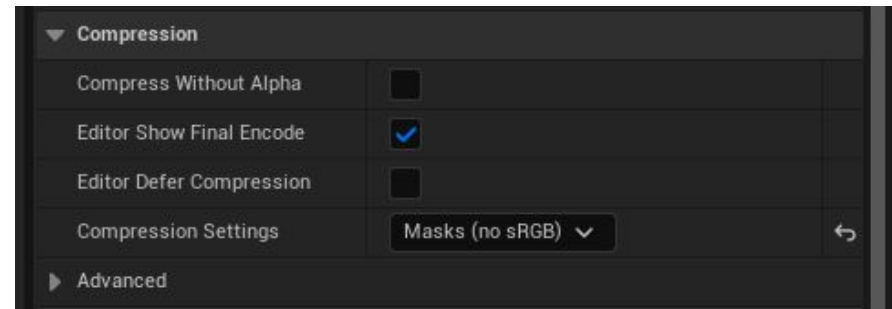
Open both the **Bread RMA** map from the **Content Browser**. Navigate to **compression settings** in the **Detail Inspector** and set **Compression Settings** to be **Masks (no sRGB)**. This will force our RMA to be treated as linear colour space. Save the changes.

Open both the **Bread Normal** map from the **Content Browser**. Navigate to **Advanced** in the **Detail Inspector** and click the **drop-down arrow**. Find **Flip Green Channel** and make sure that the option is **ticked**. Save the changes.

With this we can move onto creating our **Master Material**

Not All Assets are the Same

Some of these settings are only needed for some assets. These settings are required for any assets created using the GIM asset pipeline taught in other units (FIT1033, FIT2098 & FIT2169)



4. Creating a Basic Master Material

A Material is a shader that needs to be compiled and executed at runtime. To avoid the number of unique shaders needed, we create what are known as **Master Materials**. These have parameters that can then be modified by instances.

Open the Content Browser, navigate to the **Materials** folder, right click and select **Material** -> **Material**. Name this new material **MM_PropMaster_01a**.

Why use the MM_ prefix?

This is UE's naming convention. The MM_ prefix means that the asset is a **Master Material**. A full list can be found here:

<https://learning.monash.edu/mod/url/view.php?id=4033975>

Open the material. This will open the **Material Graph Editor**. For now we need to **create 3 parameters** and connect them to the **output**. Create the following nodes by **right clicking** and **typing the name** in the search bar.

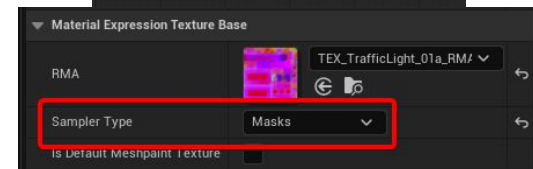
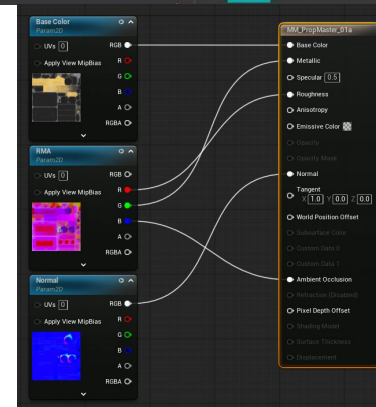
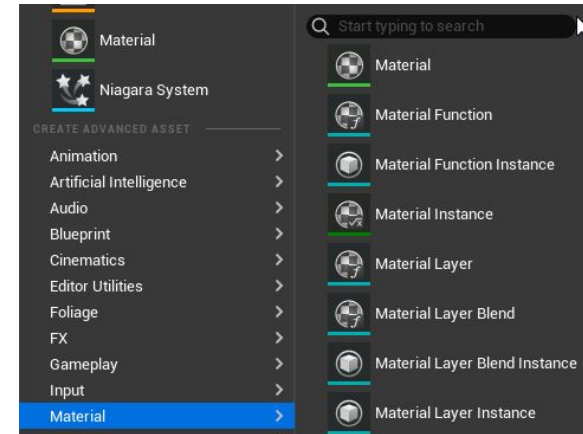
Create a **TextureSampleParameter2D**, name it **Base Color** and attach its output to the **Base Color** on the **result node**.

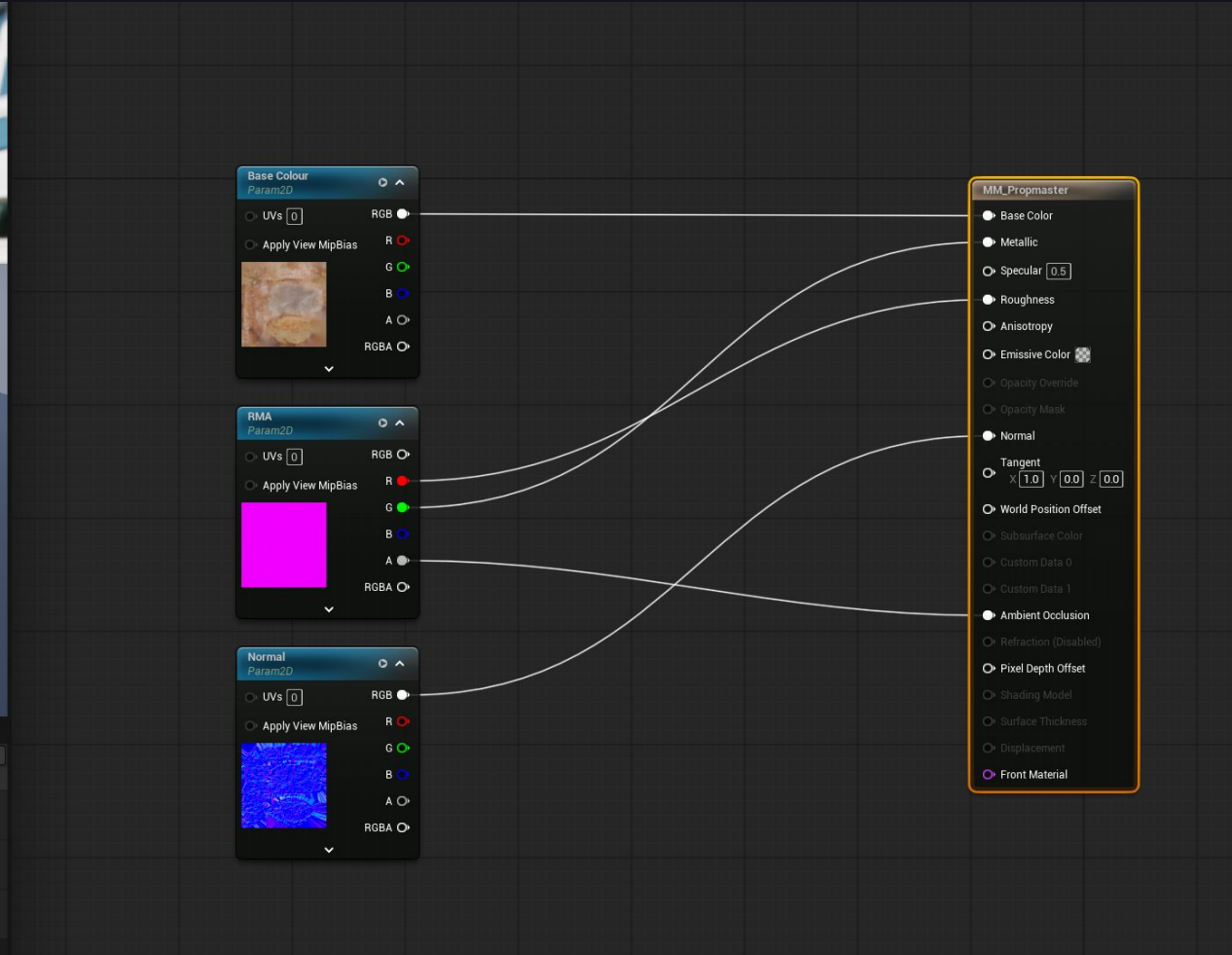
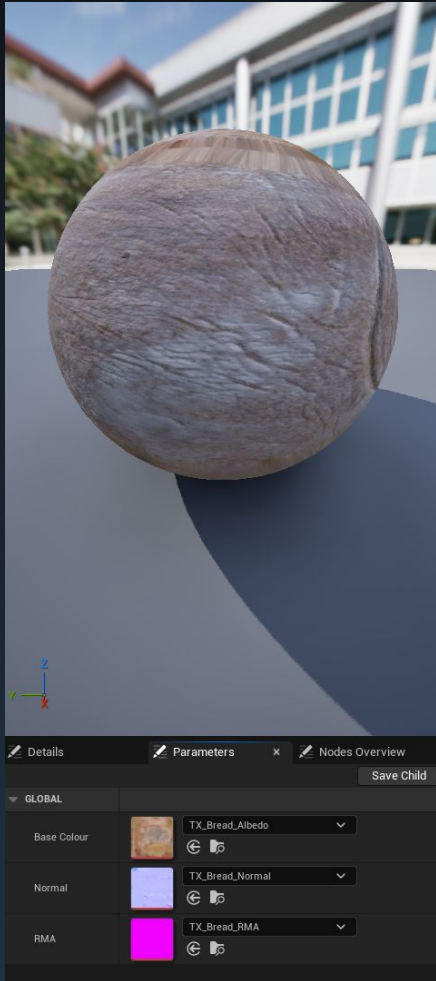
Create a **TextureSampleParameter2D**, name it **RMA**. Attach the **R** value to **Roughness**, **G** to **Metallic** and **B** to **Ambient Occlusion**. Select the node and in the **details inspector** on the left, change **Sampler Type** to **Masks**.

Create a **TextureSampleParameter2D**, name it **Normal** and attach its output to the **Normal** on the **result node**.

Create a **TextureCoordinate** node and **attach its output to the UV input** on each of the **three TextureSampleParameter2D** nodes.

Select **Apply** and **Save** from the top menu





5. Creating Material Instances

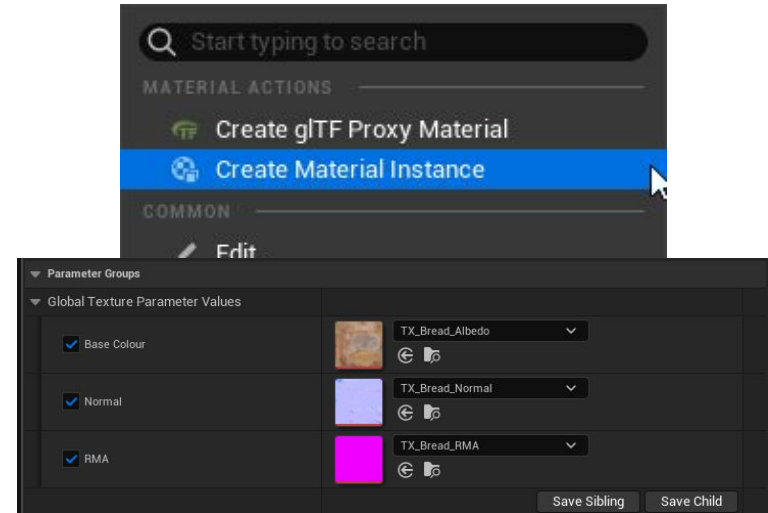
This master material is designed to be a template, not used directly on a mesh. Instead we need to create **material instances** to use. These can then be customised to suit individual meshes.

Generally each object will have one or more of its own **material instances**, depending on how it is broken up. Open the **content browser** right click on the **master material** and select **Create Material Instance**. This enables you to create a new instance that can then be adjusted as needed for each object.

You can double click on a **material instance** to bring up the instance editor. On the right hand side you will see the **details panel**. Click the arrow next to **Global Texture Parameter Values** to show the exposed parameters.

Click the **checkbox** next to each of the **Base Colour**, **Normal** and **RMA** respectively to **enable overriding**. This allows us to change the textures for this specific instance.

An example is provided to the right using the bread



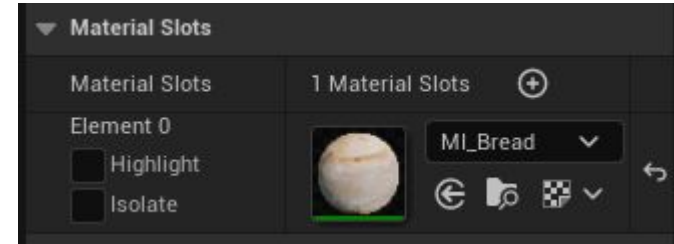
6. Testing our Material Instances

Whilst we still have more to do with our Master Material, we can still test how it is working currently. Open the content browser and open the imported bread mesh.

In the details panel on the right hand side you will notice a material slot. Set **Element 0** to be our **Bread** material instance. Once done we can see how the material looks on our bread. From the **content browser**, select and drag a **loaf of bread** into the **viewport** to add it to the scene. Place it in the centre of the world.

There is more we can do for these materials though. Sometimes we wont have roughness or normal. Sometimes we may also want to apply tints to customise more.

We need to add these in, and whilst we do we will add more parameters to make the master material even more flexible.

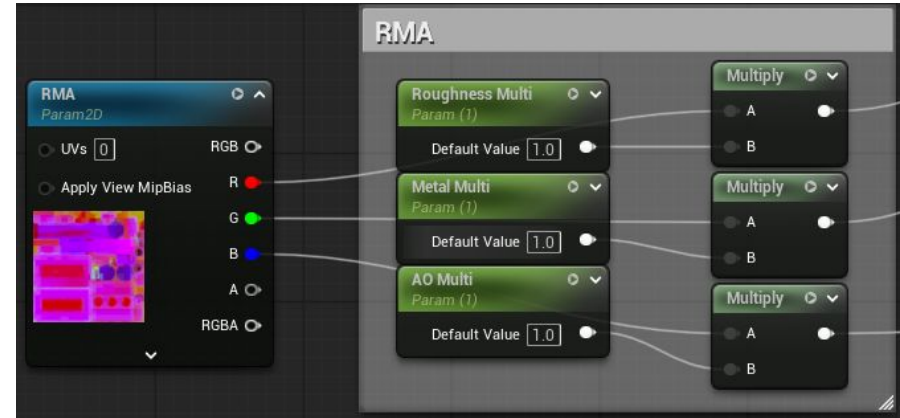


7. Controlling RMA with Parameters

Sometimes we want to be able to adjust the roughness, metallic and ambient occlusion channels without having to make changes to the texture itself. For each of the three, complete the following:

- Create a **new Scalar Parameter** node called **Roughness Multi**
- Create a new **Multiply Node**
- Connect the **Roughness Channel** to the **Multiply Node's A**
- Connect the **Roughness Multi** to the **Multiply Node's B**
- Connect the **Multiply Node output** to **Roughness** on the **Result Node**
- Select **all three** of the **scalar parameter and multiply node's** and press **C** on your keyboard. This will create a **comment** around the block. Set the comment to be **"RMA"**

Repeat the process for **Metallic** and **Ambient Occlusion** respectively. Name the scalar values **Metal Multi** and **AO Multi** respectively.



8. Adding a Tint to Base Colour

Being able to quickly add a tint to the Base Colour is a common operation for materials. Let's set that up now.

- Create a new **VectorParameter** node and name it **Tint**
- Create a new **StaticSwitchParameter** node and name it **Use Tint?**
- Create a new **Multiply** node
- Connect the **Tint's** top output to the **Multiply A** node
- Connect the **Base Colour's** RGB to the **Multiply B** node
- Connect the **Multiply output** to the **Use Tint's** **True** input node
- Connect the **Base Colour's** RGB to the **Use Tint's** **False** input node
- Connect the **Use Tint's** output to the **Result Node's** **Base Color** node.

Keyboard Shortcuts

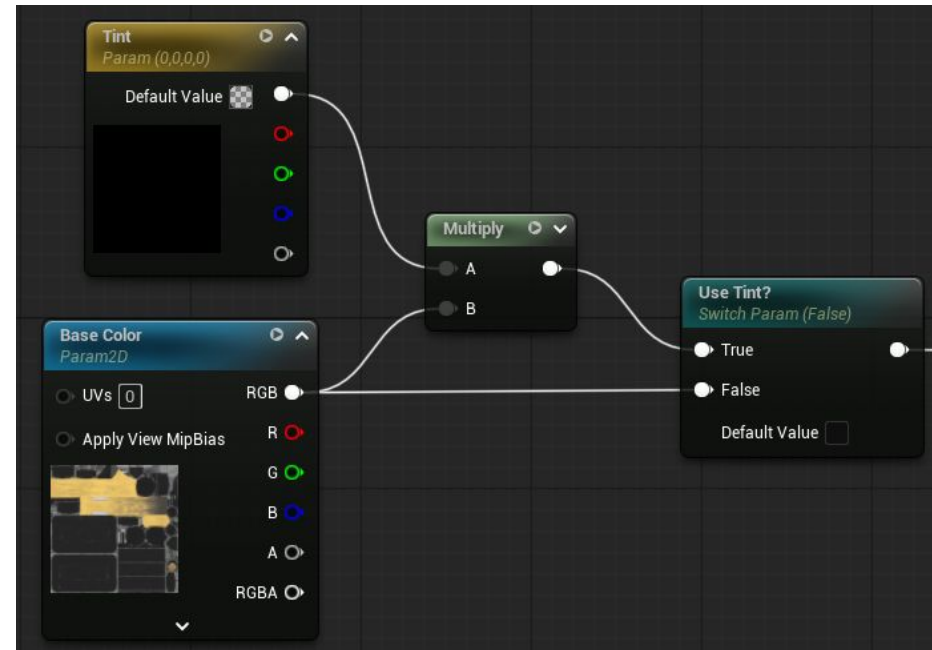
Unreal has a number of shortcuts for creating common nodes by holding down a key while left clicking.

S: Scalar Parameter

M: Multiply Node

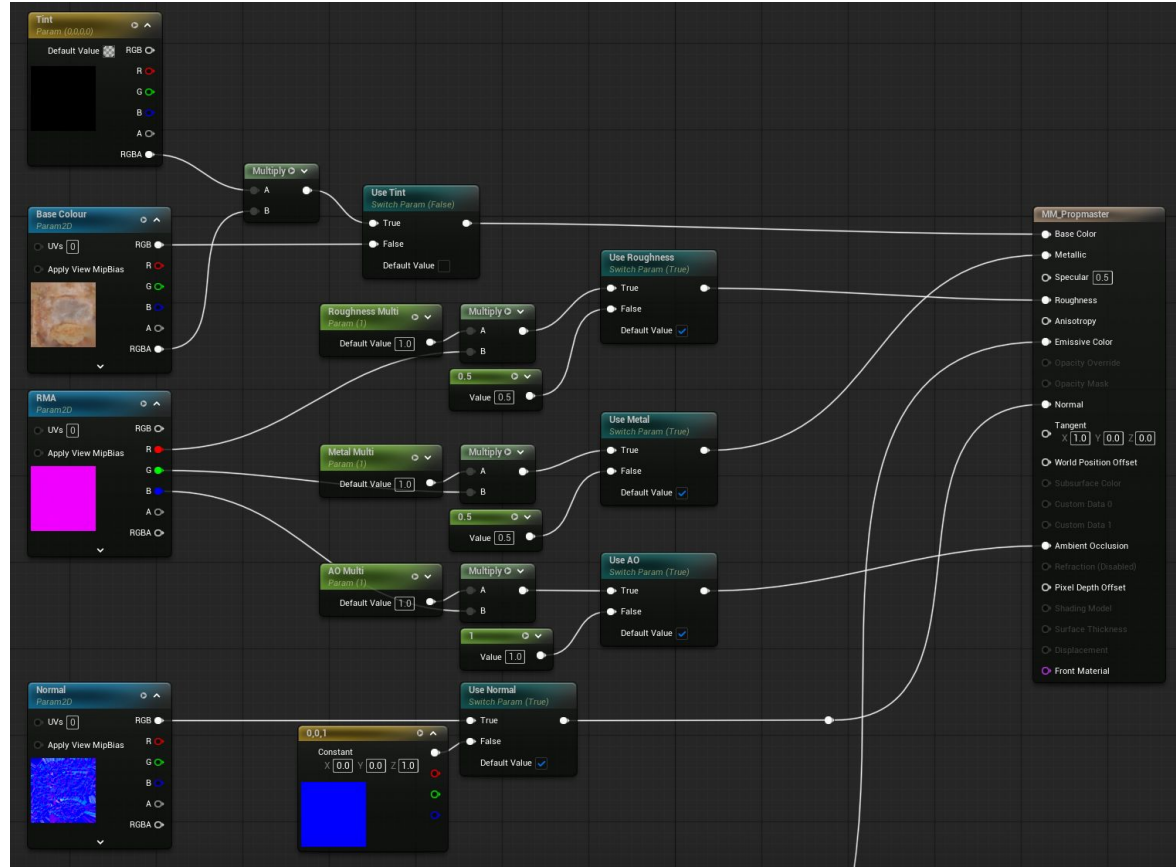
T: Texture Sample Parameter

V: Vector Parameter



9. Adding Ability to use Defaults

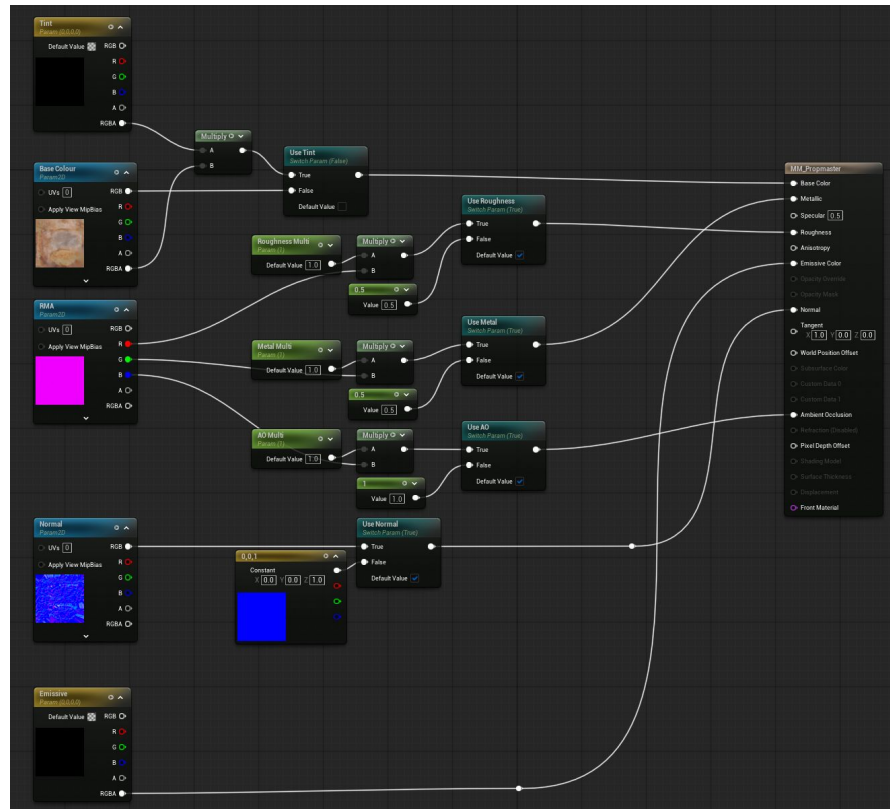
Not all assets need an RMA and a Normal Map. Let's make those static switch parameters as well



10. Adding an Emissive Channel

Not all assets will need an emissive channel, but having an emissive set to all 0 is the same as not having one at all.

Create a new **VectorParameter** and call it **Emissive**. Make sure its set to 0,0,0,0 and connect its RGBA output to Emissive on the Material.



11. Material Parameter Collections

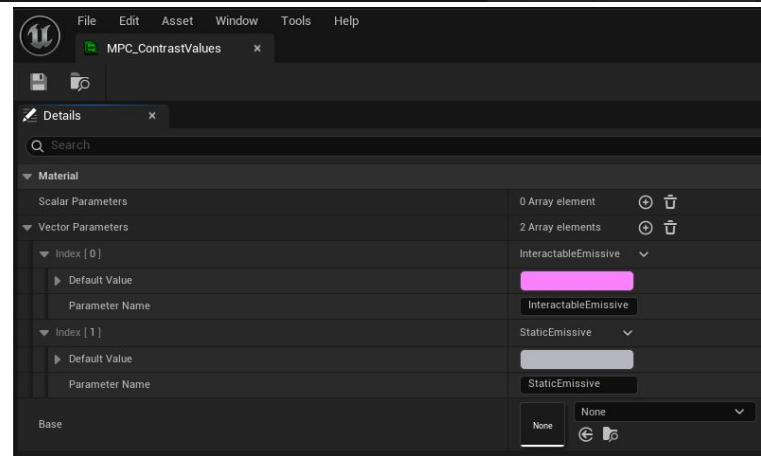
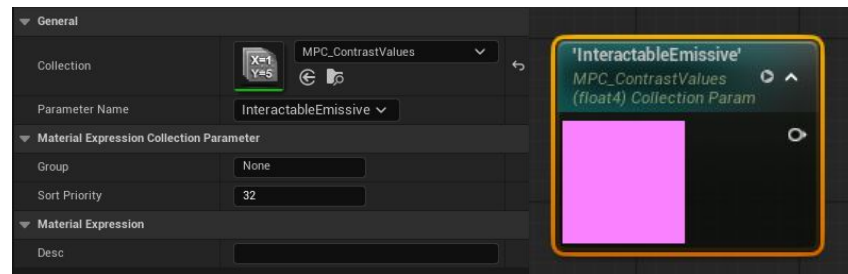
If you ever need to control a lot of values across a number of materials it may be worth using a Material Parameter Collection.

These allow you to set values in a single place and have all materials use them. There are two ways to handle this. The first way is to reference a Material Parameter Collection directly inside a Material using a Parameter Collection Node.

The second option is to reference one in code and use it to apply to Materials directly. Let's take a look at this method.

Inside the **Materials** Folder in the **Content Browser**, right click and create a new **Material Parameter Collection**. Name it **MPC_ContrastValues**.

Inside the collection you can create as many Scalar and Vector parameters as you want. For now create two Vector Parameters called **InteractableEmissive** and **StaticEmissive** and give them colour values.



12. Creating a Contrast Controller Actor

Create a new C++ class called **ContrastControllerActor** and make sure it is a sub-class of **Actor**.

Inside of the header file add the following under public:

```
void ApplyContrast();
```

```
UPROPERTY(EditAnywhere, BlueprintReadWrite)
UMaterialParameterCollection* ContrastParameterCollection;
```

```
UMaterialParameterCollectionInstance* ContrastParameterCollectionInstance;
```

Make sure to generate a definition for ApplyContrast and move over to the CPP file

12. Creating a Contrast Controller Actor (contd.)

Inside the **ApplyContrast** function add the following code:

```
if (!ContrastParameterCollection)
    return;

FLinearColor InteractableColor;
ContrastParameterCollectionInstance = GetWorld()→GetParameterCollectionInstance(ContrastParameterCollection);
ContrastParameterCollectionInstance→GetVectorParameterValue(FName("InteractableEmissive"), InteractableColor);

TArray<AActor*> FoundActors;
UGameplayStatics::GetAllActorsWithTag(GetWorld(), "Interactable", FoundActors);

for (AActor* FoundActor : FoundActors)
{
    AStaticMeshActor* SMActor = Cast<AStaticMeshActor>(FoundActor);

    UMaterialInstanceDynamic* MaterialInstanceDynamic =
        UMaterialInstanceDynamic::Create(SMActor→GetStaticMeshComponent()→GetMaterial(0), this);

    MaterialInstanceDynamic→SetVectorParameterValue("Emissive", InteractableColor);
    SMActor→GetStaticMeshComponent()→SetMaterial(0, MaterialInstanceDynamic);
}
```

Firstly, we are getting an instance of the Parameter Collection so we can use it as we see fit. Making it an instance lets us also modify values if we choose.

The next thing we are doing is getting all actors in the world with the tag "Interactable". We have not set anything yet but will soon.

For each actor found we get access to its material and change the Vector Parameter value that represents the emissive. We then set it back.

12. Creating a Contrast Controller Actor (contd.)

The final thing to do in this C++ file is to add the ApplyContrast function call to BeginPlay.

Save, compile and return to Unreal Engine

13. Creating a Blueprint Subclass

Create a Blueprint Subclass and call it BP_ContrastController Actor. Inside the properties make sure to set the Material Parameter Collection. Save and drag one into the world.

14. Using Actor Tags

All Actors have what are known as Tags that can be set either in blueprints or when they are placed into the game world. These are easiest found by searching for Tag in the details panel.

You can add as many Tags as you like to an Actor and these can be useful for filtering out specific Actors and Objects. Inside our code we are looking for Actors Tagged with Interactable.

Drag a number of Bread meshes into the scene, making sure they have a material set up previously. Add the tag “Interactable” to each of them.

Save everything, compile and play. You will notice that all the bread is now emitting the same colour you set in your Material Parameter Collection.

Consider how this could be expanded for a number of different types of objects / contrasts...

